

Red Hat npm Supply Chain Compromise

Miasma / Mini Shai-Hulud Variant Targeting @redhat-cloud-services

Lost Boys Cyber / RAlccoon Threat Intelligence Report

Incident	Miasma / @redhat-cloud-services npm compromise
Date	2026-06-01
TLP	TLP:AMBER
Confidence	High for compromise and behavior; medium for attribution
Priority	Critical for exposed CI/CD and developer environments

Report Type: Threat Intelligence / Software Supply Chain Incident

Incident Name: Miasma / Mini Shai-Hulud Variant Targeting @redhat-cloud-services

Prepared For: Lost Boys Cyber / RALccoon Reports

Prepared By: RALccoon Analyst

Date: 2026-06-01

TLP: TLP:AMBER

Confidence: High for observed package compromise and malware behavior; medium for actor attribution.

AI Disclosure: This report was drafted with AI assistance and reviewed against public vendor and security research sources available on 2026-06-01. The incident is active and evolving; affected package counts, revoked versions, and live registry state may change.

Executive Summary

On 2026-06-01, multiple security research teams reported an active npm supply chain compromise affecting legitimate packages under the @redhat-cloud-services npm namespace. The campaign is tracked by researchers as **Miasma: The Spreading Blight**, a variant of the Mini Shai-Hulud credential-stealing worm family. Reporting from Wiz, SafeDep, StepSecurity, JFrog, Aikido, and Leitwacht indicates that attackers abused GitHub Actions trusted publishing / OIDC workflows associated with Red Hat Cloud Services repositories to publish malicious package versions that carried valid npm provenance attestations.

The incident is strategically important because the malicious artifacts were not typosquats. They were official namespace packages with apparently legitimate signed provenance. The malware executed through preinstall hooks during npm install, before application code import, and targeted developer workstations, CI/CD runners, GitHub identities, npm publish tokens, cloud credentials, Kubernetes service account tokens, Vault secrets, Docker credentials, SSH material, and developer-tool configurations.

RALccoon assesses this incident as a **critical software supply chain event** for organizations that installed affected @redhat-cloud-services packages on or after 2026-06-01, especially inside CI/CD systems or developer workstations with access to cloud, GitHub, npm, or production deployment secrets. Any affected install environment should be treated as potentially compromised until forensic review proves otherwise.

Key Judgments

Judgment	Confidence	Rationale
Legitimate @redhat-cloud-services packages were compromised and republished with malicious install-time code.	High	Multiple independent researchers reported package-level artifacts, versions, hashes, and runtime behavior.
The compromise abused GitHub Actions trusted publishing / OIDC, producing valid npm provenance for malicious packages.	High	SafeDep, Wiz, Aikido, and Leitwacht described throwaway branches, orphan commits, workflow identity abuse, and valid SLSA provenance.
The payload is a Mini Shai-Hulud-derived worm variant called Miasma.	Medium-High	Payload markers and behavior align strongly with Shai-Hulud family tradecraft, but public release of tooling creates copycat risk.
Exposure impact extends beyond npm dependencies into cloud, GitHub, CI/CD, and developer endpoint compromise.	High	Malware capabilities include broad secret collection, cloud metadata access, GitHub dead-drop exfiltration, npm republishing, and persistence.
Package reputation and signed provenance alone are insufficient controls.	High	The attack produced artifacts that were both legitimate-scope and provenance-valid while still malicious.

Incident Overview

Researchers observed malicious versions of @redhat-cloud-services npm packages published on 2026-06-01. Public reporting varies between 31 and 32 affected packages due to incident churn, unpublishing, and re-arming waves. Wiz reported at least 32 package releases and roughly 80,000 combined weekly downloads. SafeDep reported 32 packages and 64 malicious versions, with a second wave still live as latest at the time of its publication. Leitwacht reported repeated re-arming at least four times by 2026-06-01 14:25 UTC.

The attackers appear to have pushed short-lived oidc-<hex> branches or orphan commits into RedHatInsights repositories, including:

- RedHatInsights/javascript-clients
- RedHatInsights/frontend-components
- RedHatInsights/platform-frontend-ai-toolkit

Those branches contained malicious publishing workflows and payload scripts such as _index.js. Because npm trusted publishing binds identity to repository and workflow filename rather than necessarily to a protected release branch, the malicious workflow could request a GitHub OIDC token, exchange it for npm publish authority, and publish tampered packages with valid provenance.

Timeline

Date / Time	Event
2026-04 to 2026-05	Mini Shai-Hulud-style supply chain activity affects multiple ecosystems, including Bitwarden CLI, SAP-related npm packages, PyTorch Lightning, TanStack/Mistral-linked waves, and Microsoft Durable Task reporting according to Aikido and related researchers.
2026-06-01 10:53 UTC	Wiz reports malicious orphan-branch activity in RedHatInsights repositories beginning around this time.
2026-06-01 10:54-10:55 UTC	Leitwacht reports first observed publish burst in its npm firehose.
2026-06-01 11:56 UTC	StepSecurity disclosure issue timing cited by Leitwacht as public reporting emerged.
2026-06-01 13:44-13:46 UTC	Wiz reports a second wave of malicious commits across RedHatInsights repositories.
2026-06-01 14:25 UTC	Leitwacht reports the scope was still being re-armed and malicious versions had valid signed provenance.
2026-06-01 15:00 UTC	Wiz adds second-wave impacted packages and additional payload details.

Source Review and Confidence

Source	Evidence Used	Confidence
Wiz Research	Campaign name, Red Hat namespace scope, estimated downloads, root cause, affected package list, IOCs, cloud identity collection additions	High
SafeDep	Trusted publishing abuse, branch/workflow mechanics, 32 packages / 64 versions claim, runtime chain, hashes, runtime artifacts	High

Source	Evidence Used	Confidence
StepSecurity	Runtime detonation in GitHub Actions, preinstall hook behavior, obfuscation chain, affected versions, CI/CD secret targeting	High
JFrog Security Research	Payload family behavior, exfiltration camouflage, persistence paths, repository/workflow IOCs, remediation order	High
Leitwacht	Provenance attestation analysis, orphan commit mechanics, behavioral detection logic, blast-radius interpretation	Medium-High
Red Hat Customer Portal	Historical context for supply chain compromises and Red Hat product-impact posture for earlier npm waves	Medium
Aikido	Campaign context, attribution caveats, timeline, trusted publishing bypass description	Medium-High

Technical Analysis

Initial Access and Publishing Path

The likely publishing path was not a stolen npm token in the traditional sense. Researchers reported that malicious packages were published through GitHub Actions OIDC trusted publishing. In this model, GitHub Actions can request an OIDC token and npm can exchange that identity for short-lived publish authority.

The attacker appears to have obtained push capability into RedHatInsights repositories, then created short-lived or orphan branches containing:

- a workflow file matching or abusing a trusted workflow identity such as `release.yml` or `ci.yml`
- `id-token: write` permissions
- a payload runner such as `_index.js`
- environment variables such as `OIDC_PACKAGES`, `WORKFLOW_ID`, and `REPO_ID_SUFFIX`
- package lists under the `@redhat-cloud-services` scope

The result was a malicious publish that could still carry valid SLSA/npm provenance. The provenance was not necessarily forged; it described a real GitHub Actions workflow execution in a legitimate repository context. The security failure is that the workflow itself was attacker-controlled.

Package Tampering

The malicious package modification pattern was intentionally small at the manifest layer:

```
{
  "scripts": {
    "preinstall": "node index.js"
  }
}
```

The `preinstall` hook executes automatically during `npm install`, before the dependent project imports the package. This makes CI/CD and developer workstations high-risk even when the vulnerable package is never used at runtime.

Multiple researchers noted that `index.js` ballooned from normal library size to multi-megabyte obfuscated JavaScript, around 4.2 to 4.3 MB in analyzed samples. That file acted as an install-time loader and staging mechanism.

Loader and Staging

Reported loader characteristics include:

- ROT-style string decoding such as ROT-21 or ROT-9, depending on sample/report.
- Large numeric character arrays rebuilt with `String.fromCharCode`.
- Silent `try/catch` wrappers to reduce visible installation errors.
- AES-128-GCM encrypted payload blobs.
- Obfuscator-style string tables and dynamic `eval`.
- Bun runtime bootstrap if Bun is not present.
- Temporary payload paths such as `/tmp/p<random>.js`.
- Runtime artifacts such as `/tmp/b-<random>/bun`, `/tmp/b-<random>/bun.exe`, `/tmp/b-<random>/b.zip`, and `/tmp/.bun_ran`.

SafeDep reported one analyzed package with:

Artifact	SHA256
Tarball	031ba872d5a84bfb18115f432811e4b45180346a1bae653f7fd85f918e7bb3a3
<code>index.js</code>	df1732f5bfec12e066be44dee02ec8a243e4868d38672c1b1d065359dd735a14
Decrypted payload	0dc06ecdaa63fe24859cfd955053c23245c536e4733480239d14bebf12688e35

JFrog reported hashes for `@redhat-cloud-services/types@3.6.1`:

Artifact	SHA256
Malicious package metadata	7069e28a5806db4ab0273639667d203f5e31b401d403af7e36d9f360c1f6d655
Obfuscated install-time loader	b86c5ae9e95bd841a595440faa3eb6317441e746f241ae8fd641ab59ed1d1966

Credential Theft

The payload targets local and runtime secrets across software delivery environments:

- GitHub classic PATs, fine-grained PATs, GitHub Actions tokens, GitHub CLI tokens
- npm publish tokens and trusted publishing contexts
- AWS credentials, IMDS, ECS task role material, Secrets Manager, SSM Parameter Store
- Azure managed identity and Azure Key Vault secrets
- GCP service account metadata and GCP Secret Manager values
- Kubernetes service account tokens and cluster secrets
- HashiCorp Vault tokens
- Docker credentials
- SSH private keys
- environment variables, `.env` files, shell histories, git credentials
- password-manager materials such as Bitwarden/gopass, per researcher reporting
- CircleCI tokens and CI/CD environment secrets

Exfiltration and Dead-Drop Behavior

The campaign marker is:

```
Miasma: The Spreading Blight
```

Researchers observed or reported public GitHub repositories created with that description, used as exfiltration dead-drops. Result files may appear under paths like:

```
results/results-<timestamp>-<counter>.json
```

JFrog reported a configured destination shaped as:

```
hxxps[://api[.]anthropic[.]com/v1/api
```

JFrog assesses this as camouflage against a legitimate API host and explicitly notes it does not prove compromise of Anthropic infrastructure. Defenders should avoid over-broad blocking that breaks legitimate use, and instead hunt for the full path, unusual clients, node/bun parentage, and CI runner egress during `npm install`.

Propagation

The malware contains worm-like propagation capabilities:

- use stolen npm tokens to identify writable packages
- download legitimate package tarballs
- inject a payload and `preinstall` script
- bump patch versions and republish
- use GitHub Actions OIDC paths to obtain publish authority where available
- enumerate writable GitHub repositories
- inject suspicious workflows such as `.github/workflows/codeql.yml`
- modify JavaScript GitHub Actions to run injected Bun payloads

StepSecurity reported use of npm's `bypass_2fa` parameter in propagation attempts, creating risk even for accounts with 2FA where token scope and registry behavior allow publish actions.

Persistence and Destructive Remediation Trap

JFrog reported local persistence mechanisms and a destructive token-monitor trap:

- `~/.config/systemd/user/kitty-monitor.service`
- `~/.local/share/kitty/cat.py`
- `~/.Library/LaunchAgents/com.user.kitty-monitor.plist`
- `~/.config/gh-token-monitor/token`
- `~/.config/gh-token-monitor/handler`
- `~/.local/bin/gh-token-monitor.sh`
- `~/.config/systemd/user/gh-token-monitor.service`
- `~/.Library/LaunchAgents/com.user.gh-token-monitor.plist`
- `~/.config/index.js`
- `.claude/settings.json`
- `.claude/setup.mjs`
- `.vscode/tasks.json`

Important response order: isolate affected systems and remove persistence before revoking GitHub tokens. JFrog reports the token monitor may execute destructive commands when it detects that a stored token was invalidated.

Affected Packages

Public reporting was evolving at publication time. The following affected package/version list consolidates Wiz and StepSecurity reporting and should be validated against npm registry metadata, lockfiles, and internal package caches before remediation decisions.

Package	Reported Malicious Versions
@redhat-cloud-services/types	3.6.1, 3.6.2, 3.6.4
@redhat-cloud-services/frontend-components-utilities	7.4.1, 7.4.2, 7.4.4
@redhat-cloud-services/frontend-components	7.7.2, 7.7.3, 7.7.5
@redhat-cloud-services/rbac-client	9.0.3, 9.0.4, 9.0.6
@redhat-cloud-services/javascript-clients-shared	2.0.8, 2.0.9, 2.0.11
@redhat-cloud-services/frontend-components-config-utilities	4.11.2, 4.11.3, 4.11.5
@redhat-cloud-services/frontend-components-notifications	6.9.2, 6.9.3, 6.9.5
@redhat-cloud-services/tsc-transform-imports	1.2.2, 1.2.4, 1.2.6; Wiz also lists 1.2.3, 1.2.5
@redhat-cloud-services/frontend-components-config	6.11.3, 6.11.4, 6.11.6
@redhat-cloud-services/eslint-config-redhat-cloud-services	3.2.1, 3.2.2, 3.2.4
@redhat-cloud-services/host-inventory-client	5.0.3, 5.0.4, 5.0.6
@redhat-cloud-services/rule-components	4.7.2, 4.7.3, 4.7.5
@redhat-cloud-services/frontend-components-remediations	4.9.2, 4.9.3, 4.9.5
@redhat-cloud-services/frontend-components-translations	4.4.1, 4.4.2, 4.4.4; Wiz initially lists 4.4.1
@redhat-cloud-services/vulnerabilities-client	2.1.8, 2.1.9, 2.1.11
@redhat-cloud-services/frontend-components-advisor-components	3.8.2, 3.8.4, 3.8.6; Wiz lists 3.8.3, 3.8.5 in one table
@redhat-cloud-services/entitlements-client	4.0.11, 4.0.12, 4.0.14
@redhat-cloud-services/chrome	2.3.1, 2.3.2, 2.3.4
@redhat-cloud-services/notifications-client	6.1.4, 6.1.5, 6.1.7
@redhat-cloud-services/compliance-client	4.0.3, 4.0.4, 4.0.6
@redhat-cloud-services/sources-client	3.0.10, 3.0.11, 3.0.13
@redhat-cloud-services/integrations-client	6.0.4, 6.0.5, 6.0.7
@redhat-cloud-services/frontend-components-testing	1.2.1, 1.2.2, 1.2.4
@redhat-cloud-services/remediations-client	4.0.4, 4.0.5, 4.0.7
@redhat-cloud-services/insights-client	4.0.4, 4.0.5, 4.0.7
@redhat-cloud-services/topological-inventory-client	3.0.10, 3.0.11, 3.0.13
@redhat-cloud-services/config-manager-client	5.0.4, 5.0.5, 5.0.7
@redhat-cloud-services/hcc-pf-mcp	0.6.1, 0.6.2, 0.6.4
@redhat-cloud-services/quickstarts-client	4.0.11, 4.0.12, 4.0.14

Package	Reported Malicious Versions
@redhat-cloud-services/patch-client	4.0.4, 4.0.5, 4.0.7; Wiz lists 4.0.6 in one table
@redhat-cloud-services/hcc-feo-mcp	0.3.1, 0.3.2, 0.3.4
@redhat-cloud-services/hcc-kessel-mcp	0.3.1, 0.3.2, 0.3.4

Indicators of Compromise

Campaign and Repository IOCs

Indicator	Type	Notes
Miasma: The Spreading Blight	GitHub repository description	Attacker-created exfiltration repository marker
results/results-*.json	Repository path	Exfiltrated result file pattern
oidc-<hex>	Branch pattern	Short-lived malicious publishing branches reported by researchers
chore/add-codeql-static-analysis	Branch/workflow marker	SafeDep-reported worm fingerprint
.github/workflows/codeql.yml	Workflow path	Potential injected workflow
_index.js	Workflow payload filename	Used in malicious publish workflow
OIDC_PACKAGES	Workflow env var	Publishing/propagation logic
WORKFLOW_ID	Workflow env var	Publishing/propagation logic
REPO_ID_SUFFIX	Workflow env var	Publishing/propagation logic
VARIABLE_STORE	Workflow env var	GitHub Actions secret dumping logic
format-results	Artifact name	Possible secret dumping artifact
format-results.txt	Artifact file	Possible secret dumping artifact
Run Copilot	Workflow name	Suspicious workflow marker reported by JFrog
IfYouInvalidateThisTokenItWillNukeTheComputerOfTheOwner	Commit/message marker	Token invalidation threat marker

Network IOCs

Indicator	Type	Notes
hxxps[://api[.]anthropic[.]com/v1/api	HTTPS path	Camouflaged exfiltration destination/path reported by JFrog; legitimate host, suspicious path
hxxps[://api[.]github[.]com/search/commits?q=firedalazer	GitHub API path	Commit-search C2 used by kitty-monitor per JFrog
github.com/oven-sh/bun/releases/download/bun-v1.3.13/	Download URL prefix	Bun runtime retrieval during install chain
google-api-nodejs-client/7.0.0 gl-node/20.11.0 gccl/7.0.0	User agent	GCP querying indicator reported by Wiz

Host IOCs

Indicator	Type
/tmp/p*.js	Transient payload
/tmp/b-*/bun	Downloaded Bun runtime
/tmp/b-*/bun.exe	Downloaded Bun runtime on Windows
/tmp/b-*/b.zip	Downloaded Bun archive
/tmp/.bun_ran	Bun execution marker
/var/tmp/.gh_update_state	Kitty monitor state
~/.local/share/kitty/cat.py	Python payload
~/.config/systemd/user/kitty-monitor.service	Linux persistence
~/Library/LaunchAgents/com.user.kitty-monitor.plist	macOS persistence
~/.config/gh-token-monitor/token	Stored GitHub token
~/.config/gh-token-monitor/handler	Dead-man handler
~/.local/bin/gh-token-monitor.sh	Token monitor script
~/.config/systemd/user/gh-token-monitor.service	Linux persistence
~/Library/LaunchAgents/com.user.gh-token-monitor.plist	macOS persistence
~/.config/index.js	Developer-tool payload copy
.claude/settings.json	Possible AI-tool hook target
.claude/setup.mjs	Possible AI-tool persistence file
.vscode/tasks.json	Possible VS Code folder-open persistence
.github/setup.js	Possible repository payload

Detection Engineering

Sigma: npm Lifecycle Script Executes Suspicious Bun Loader

```

title: Suspicious npm Install Lifecycle Script Launching Bun Loader
id: 12c4566b-2b66-4d33-a3ef-miasma-npm-bun
status: experimental
description: Detects npm install chains where node executes a package index.js preinstall loader
that stages Bun and temporary JavaScript payloads.
references:
  - https://www.stepsecurity.io/blog/multiple-redhat-cloud-services-npm-packages-compromised
  - https://research.jfrog.com/post/shai-hulud-miasma-redhat-cloud-services/
logsource:
  category: process_creation
detection:
  npm_node:
    ParentImage|endswith:
      - '/npm'
      - '/npm-cli.js'
      - '\\npm.cmd'
    Image|endswith:
      - '/node'
      - '\\node.exe'
    CommandLine|contains:
      - 'index.js'
  bun_exec:
    CommandLine|contains:
      - 'bun run /tmp/p'
      - 'bun-v1.3.13'
      - '/tmp/b-'
suspicious_package:

```

```

      CommandLine|contains:
        - '@redhat-cloud-services'
        - 'node index.js'
    condition: npm_node or bun_exec or suspicious_package
  fields:
    - Image
    - ParentImage
    - CommandLine
    - CurrentDirectory
    - User
  falsepositives:
    - Legitimate npm lifecycle scripts that use Bun. Tune with package name, path, and hash
  allowlists:
  level: high

```

Sigma: GitHub Actions Workflow OIDC Publish Abuse

```

title: Suspicious GitHub Actions OIDC npm Publish Workflow
id: 3bc0c73e-59f1-4d57-9c81-miasma-oidc-publish
status: experimental
description: Detects suspicious workflow changes that grant id-token write and run Bun payloads
for npm package publishing.
references:
  - https://safedep.io/redhat-cloud-services-hit-by-mini-shai-hulud-npm-worm/
  - https://leitwacht.eu/blog/valid-provenance-malicious-package
logsource:
  product: github
  service: audit
detection:
  workflow_change:
    action|contains:
      - 'workflow'
      - 'push'
      - 'repo'
  suspicious_content:
    file_path|contains:
      - '.github/workflows/'
    content|contains:
      - 'id-token: write'
      - 'bun run _index.js'
      - 'OIDC_PACKAGES'
      - 'WORKFLOW_ID'
      - 'REPO_ID_SUFFIX'
  suspicious_branch:
    ref|re: 'refs/heads/oidc-[a-f0-9]+'
  condition: suspicious_content or suspicious_branch
  fields:
    - actor
    - repository
    - ref
    - file_path
    - action
  falsepositives:
    - Legitimate trusted publishing workflows. Require protected branch, release/tag context, and
    expected workflow file hash.
  level: high

```

Splunk SPL: Developer or CI Host Miasma Behavior

```

index=edr OR index=osquery
(
  CommandLine="*node index.js*" OR
  CommandLine="*bun run /tmp/p*" OR
  CommandLine="*bun-v1.3.13*" OR
  CommandLine="*gh auth token*" OR
  CommandLine="*api.anthropic.com/v1/api*" OR
  CommandLine="*api.github.com/search/commits?q=firedalazer*"
)
| eval host_role=case(match(host,"runner|build|ci|gha"),"ci_runner",true(),"developer_or_unknown")
| stats min(_time) as first_seen max(_time) as last_seen values(CommandLine) as commands values(
ParentImage) as parents by host user host_role
| convert ctime(first_seen) ctime(last_seen)

```

Microsoft Sentinel KQL: CI Runner Egress and Process Hunt

```

let suspicious_paths = dynamic(["/v1/api", "/search/commits?q=firedalazer"]);
DeviceProcessEvents
| where ProcessCommandLine has_any ("node index.js", "bun run /tmp/p", "bun-v1.3.13", "gh auth
token", "OIDC_PACKAGES", "gh-token-monitor")
| project Timestamp, DeviceName, AccountName, FileName, ProcessCommandLine,
InitiatingProcessFileName, InitiatingProcessCommandLine
| union (
  DeviceNetworkEvents
  | where RemoteUrl has_any ("api.anthropic.com", "api.github.com", "github.com/oven-sh/bun")
  | where RemoteUrl has_any (suspicious_paths) or InitiatingProcessCommandLine has_any ("node",
"bun", "npm", "yarn", "pnpm")
  | project Timestamp, DeviceName, AccountName, FileName=InitiatingProcessFileName,
ProcessCommandLine=InitiatingProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessCommandLine
)

```

GitHub Audit Hunt

Search GitHub audit logs and repository history for:

- branch creations matching oidc-*
- workflow edits adding id-token: write
- workflow execution from non-default branches using trusted publishing
- commits adding _index.js, .github/setup.js, .github/workflows/codeql.yml
- repositories created with description "Miasma: The Spreading Blight"
- artifacts named format-results or format-results.txt
- commits/messages containing "IfYouInvalidateThisTokenItWillNukeTheComputerOfTheOwner"

Threat Hunting Plan

Dependency Exposure

1. Search all package-lock.json, npm-shrinkwrap.json, pnpm-lock.yaml, yarn.lock, SBOMs, npm caches, and container build logs for @redhat-cloud-services.
2. Match exact affected versions listed in this report against installed artifacts.
3. Include ephemeral CI runners and cached container layers, not just current source repositories.
4. Treat npm install, npm ci, pnpm install, and yarn install on 2026-06-01 as execution events if affected versions were resolvable.

Endpoint and CI Runner Hunt

Hunt for:

- node index.js launched from npm package directories
- bun run /tmp/p*.js
- downloads from github.com/oven-sh/bun/releases/download/bun-v1.3.13/
- temporary files under /tmp/p*.js and /tmp/b-*
- gh auth token
- reads of .npmrc, .env, .aws, .ssh, .docker, .kube, shell histories, and cloud config paths by node/bun
- persistence paths listed in this report
- .claude, .vscode, Codex/Gemini/Copilot/Kiro/opencode config modifications

Cloud and Identity Hunt

Hunt for:

- unusual AWS STS calls, Secrets Manager reads, SSM Parameter Store reads, or IMDS access from CI/developer networks
- Azure Managed Identity or Key Vault access from build hosts
- GCP metadata and Secret Manager access with user agent google-api-nodejs-client/7.0.0 gl-node/20.11.0 gcc/7.0.0
- Kubernetes secret list/get activity from build runner service accounts
- GitHub token use from new IPs after package installation time
- npm publish actions for unexpected patch versions or lifecycle script additions

MITRE ATT&CK Mapping

Technique	Name	Relevance
T1195.001	Supply Chain Compromise: Compromise Software Dependencies and Development Tools	Malicious code delivered through npm package dependency chain

Technique	Name	Relevance
T1059.007	Command and Scripting Interpreter: JavaScript	Node/Bun JavaScript loader and payload execution
T1059.004	Unix Shell	Shell commands used for staging, persistence, and destructive actions
T1552.001	Unsecured Credentials: Credentials In Files	Collection of .env, config files, shell histories, cloud credentials
T1552.005	Unsecured Credentials: Cloud Instance Metadata API	AWS/Azure/GCP metadata collection
T1528	Steal Application Access Token	GitHub, npm, CI/CD, and cloud token theft
T1119	Automated Collection	Broad automated credential and environment collection
T1105	Ingress Tool Transfer	Bun runtime download and follow-on payload retrieval
T1041	Exfiltration Over C2 Channel	Result exfiltration to GitHub repositories and camouflaged HTTPS path
T1098.004	Account Manipulation: SSH Authorized Keys	Potential key and repository access abuse after credential theft
T1505.003	Server Software Component: Web Shell	Not directly observed; included only for downstream web-deployment review if stolen CI credentials deploy code
T1578	Modify Cloud Compute Infrastructure	Possible follow-on if cloud credentials are valid
T1608.001	Stage Capabilities: Upload Malware	Repackaging and publishing malicious packages
T1027	Obfuscated Files or Information	Multi-layer ROT/AES/string-table obfuscation

Remediation and Incident Response

Immediate Containment

1. Freeze automated dependency updates and package publishes for affected environments.
2. Identify every system that installed affected versions on or after 2026-06-01.
3. Isolate affected developer machines and CI/CD runners before revoking GitHub tokens.
4. Preserve forensic evidence: disk images, npm caches, CI job logs, process telemetry, shell history, audit logs, package-lock files, container images, and GitHub/npm audit logs.
5. Remove persistence and token-monitor components before credential revocation.

Credential Rotation

After isolation and persistence removal, rotate:

- GitHub PATs, fine-grained PATs, GitHub App keys, deploy keys, SSH keys
- npm tokens and organization publish credentials
- GitHub Actions secrets and environment secrets
- AWS access keys, role sessions, Secrets Manager and SSM-exposed secrets

- Azure service principals, managed identity secrets, Key Vault secrets
- GCP service account keys, workload identity bindings, Secret Manager secrets
- Kubernetes service account tokens and kubeconfigs
- Vault tokens and AppRole secrets
- Docker registry credentials
- password-manager tokens exposed to affected hosts

Build System Recovery

1. Rebuild CI runners from clean images.
2. Invalidate package manager caches that contain affected tarballs.
3. Regenerate lockfiles from trusted package metadata.
4. Use `npm ci --ignore-scripts` temporarily where lifecycle scripts are not required.
5. Implement a quarantine period for newly published versions of sensitive packages.
6. Require protected branch/tag/release conditions for trusted publishing.
7. Pin trusted publishing to exact workflow files and protected refs where supported.
8. Alert when a package gains a new lifecycle script, especially `preinstall`.
9. Alert when official-scope package tarball size changes sharply between adjacent versions.
10. Treat provenance as traceability, not as a malware verdict.

Recommended Controls

Control	Priority	Purpose
Dependency allowlisting for production builds	Immediate	Prevent unreviewed package/version drift
Lifecycle script review and default blocking	Immediate	Stop <code>preinstall</code> execution paths unless explicitly approved
SBOM and lockfile integrity enforcement	Immediate	Identify and prevent affected versions
CI egress allowlisting	Immediate	Prevent arbitrary API exfiltration from build runners
Ephemeral, least-privilege CI credentials	Immediate	Limit value of stolen runner secrets
GitHub protected branch-only publishing	High	Prevent throwaway branch trusted publishing abuse
Package immaturity/quarantine policy	High	Delay adoption of just-published versions
Runtime EDR on CI runners	High	Detect credential theft and staging behavior
GitHub/npm audit log centralization	High	Trace malicious publish and workflow events
Secret scanning plus rapid revocation playbooks	High	Reduce post-compromise dwell time

Analyst Assessment

Miasma is a direct warning that software supply chain defenses cannot stop at namespace trust, maintainer reputation, or signed provenance. In this incident, the artifacts were legitimate-scope packages and the provenance appears to have truthfully attested to a

workflow in a real Red Hat repository context. The maliciousness lived in the workflow and package behavior, not in an obviously fake publisher identity.

The most important response pivot is environment exposure. If an affected version reached a CI runner, developer workstation, or container build context with useful secrets, the incident should be handled as credential compromise, not simply dependency cleanup. The payload's ability to collect tokens, query cloud metadata, create GitHub dead-drop repositories, mutate workflows, and republish packages gives it both espionage and propagation value.

Attribution should remain cautious. The payload and campaign markers strongly overlap with Mini Shai-Hulud / TeamPCP tradecraft, but public release of Mini Shai-Hulud tooling means a copycat actor can reproduce the mechanics. Operationally, attribution is secondary to containment, forensic preservation, and credential rotation.

References

1. Wiz Research, "Miasma: Supply Chain Attack Targeting RedHat npm Packages", 2026-06-01:
<https://www.wiz.io/blog/miasma-supply-chain-attack-targeting-redhat-npm-packages>
2. SafeDep, "Mini Shai-Hulud 'Miasma: The Spreading Blight' Hits @redhat-cloud-services", 2026-06-01:
<https://safedep.io/redhat-cloud-services-hit-by-mini-shai-hulud-npm-worm/>
3. StepSecurity, "Multiple redhat-cloud-services npm Packages compromised", 2026-06-01:
<https://www.stepsecurity.io/blog/multiple-redhat-cloud-services-npm-packages-compromised>
4. JFrog Security Research, "Shai-Hulud - Miasma: The Spreading Blight Hits Red Hat npm Packages", 2026-06-01:
<https://research.jfrog.com/post/shai-hulud-miasma-redhat-cloud-services/>
5. Leitwacht, "Valid provenance, malicious package: anatomy of the Red Hat npm compromise", 2026-06-01:
<https://leitwacht.eu/blog/valid-provenance-malicious-package>
6. Aikido, "Red Hat npm Packages Compromised to Spread a Credential-Stealing Worm", 2026-06-01:
<https://www.aikido.dev/blog/red-hat-npm-packages-compromised-credential-stealing-worm>
7. Red Hat Customer Portal, "Multiple Supply Chain Attacks against npm Packages":
<https://access.redhat.com/security/supply-chain-attacks-NPM-packages>
8. Red Hat Customer Portal, RHSB-2026-001 "Multiple supply chain compromises of open source projects":
<https://access.redhat.com/security/vulnerabilities/RHSB-2026-001>